

IAM Policy Scraping Report

Run Details

Field	Value
Date	2025-05-24
Script	scripts/scrape_iam_terraform.py
Pages per query	3 (30 results/page)
Total queries	4
GitHub authentication	Token (30 req/min)
Total disk usage	~3.9 MB

Queries Used

1. `aws_iam_policy_document extension:tf`
 2. `aws_iam_role_policy extension:tf`
 3. `iam:PassRole extension:tf`
 4. `aws_iam_policy extension:tf resource`
-

Results

Metric	Count
Total files scraped	959
JSON policies	0
Raw TF blocks	959
Unique policies (deduplicated)	959

Why 0 JSON and 959 raw TF?

The `python-hcl2` parser failed on most files because GitHub Terraform configs typically use HCL interpolation expressions (`${var.xxx}` , `replace(...)` , etc.) that the hcl2 library cannot parse. The regex fallback successfully captured all IAM blocks as raw HCL text snippets.

Resource Type Distribution

Resource Type	Count
aws_iam_policy_document	305
aws_iam_role	291
aws_iam_policy	221
aws_iam_role_policy	135
aws_iam_user_policy	6
aws_iam_group_policy	1

Dangerous Patterns Detected

Pattern	Files
iam:PassRole	103
sts:AssumeRole	273
Wildcard actions (Action = "*")	47
iam:* (full IAM admin)	3
iam:AttachRolePolicy / AttachUserPolicy	9
iam:PutRolePolicy / PutUserPolicy	8
iam:CreatePolicy	2

Top IAM Actions

Action	Occurrences
iam:PassRole	106
iam:CreateServiceLinkedRole	12
iam:GetRole	15
iam:GetInstanceProfile	11
iam:CreateRole	7
iam:AttachRolePolicy	6
iam:PutRolePolicy	5
iam:PutUserPolicy	6

Sample Policies

Sample 1: WAF Log Policy Document (secure)

```
data "aws_iam_policy_document" "waf" {
  version = "2012-10-17"
  statement {
    effect = "Allow"
    principals {
      identifiers = ["delivery.logs.amazonaws.com"]
      type       = "Service"
    }
    actions       = ["logs:CreateLogStream", "logs:PutLogEvents"]
    resources     = ["${aws_cloudwatch_log_group.waf.arn}:*"]
    condition {
      test      = "ArnLike"
      values    = ["arn:aws:logs:${var.region}:..."]
      variable  = "aws:SourceArn"
    }
  }
}
```

Sample 2: Overly Permissive Policy (vulnerable)

```
resource "aws_iam_policy" "bad_policy" {
  name       = "bad_policy"
  description = "A terrible idea."
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = "*"
        Resource = "*"
      }
    ]
  })
}
```

Sample 3: PassRole + ECS RunTask (potentially vulnerable)

```
data "aws_iam_policy_document" "lambda_start_yolo_task" {
  statement {
    sid      = "RunYoloTask"
    actions = ["ecs:RunTask"]
    resources = [replace(aws_ecs_task_definition.yolo_detect_worker.arn, ...)]
  }
  statement {
    sid      = "PassYoloRoles"
    actions = ["iam:PassRole"]
    resources = [
      aws_iam_role.yolo_detect_worker_task.arn,
      aws_iam_role.yolo_detect_worker_execution.arn,
    ]
  }
}
```

Dataset Expansion Impact

Metric	Before	After (potential)
Total policies	108	108 + 959 = 1,067
Vulnerable samples (est.)	41	~41 + 150 = ~191*
Secure samples (est.)	67	~67 + 809 = ~876*
Coverage increase	—	~9.9x

* Estimated. Scraped policies require manual labeling before use in training.

Next Steps

1. **Manual labeling needed** — Scraped policies must be reviewed and labeled as `vulnerable` or `secure` before they can be used for GNN training.
 2. **HCL → JSON conversion** — Raw `.tf` blocks need to be converted to the PolicyGraph JSON format (see `scripts/integrate_scraped.py`).
 3. **Quality review** — Some blocks may be incomplete or contain only role definitions without policy statements.
 4. **Incremental scraping** — Run with `--pages 10` for ~3,000+ policies, or use different queries for broader coverage.
 5. **Integration into training dataset** — Use `scripts/integrate_scraped.py` to merge labeled policies into `data/raw/samples/`.
-

Rate Limit Notes

- **With token:** 30 search requests/minute, 5,000 requests/hour
- **Without token:** 10 search requests/minute, 60 requests/hour
- The script includes automatic rate-limit handling (backs off on 403)
- A 2-second delay between pages is built in